NUMERICAL ANALYSIS
GUIDED LAB 3

ROB TOMPKINS

Note, all the files associated with Guided Lab 3 are checked into the :
https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab3

Problems 1, 2, and 3, were predominantly only writing code. The code can be found at: naturalCubicSplines.m. Though, the code for these three problems also follows as

```
function [A,rvec,c,a,b,d] = naturalCubicSplines(x,y)
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Natural cubic spline interpolator
    %
    % Inputs:
    %   x     - vector of x data points
    %   y     - vector of y data points
    %   xstar - point(s) where spline is to be evaluated
    %
    % Output:
    %   ystar - spline value(s) at xstar
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    if length(x) < 2 | length(y) < 2
        error('Input vectors are not long enough, please make them of length two or more and be of the same size')
    end
    if length(x) ~= length(y)
        error('Input vectors are not the same length')
    end

    % Make sure x and y are column vectors
    x = x(:);
    y = y(:);

    % n = number of subintervals
    n = length(x) - 1;

    % Initialize A and rvec
    A = zeros(n+1,n+1);
    rvec = zeros(n+1,1);

    % Calculate vector of h values
    h = diff(x);

    % Natural spline boundary conditions
    A(1,1) = 1;
    A(n+1,n+1) = 1;
```

Date: March 16, 2026.

```
    % Populate rest of matrix A and RHS vector
    for i = 2:n
        A(i,i-1) = h(i-1);
        A(i,i)   = 2*(h(i-1) + h(i));
        A(i,i+1) = h(i);
        rvec(i)  = 3*((y(i+1)-y(i))/h(i) - (y(i)-y(i-1))/h(i-1));
    end

    % Solve for c coefficients
    c = A \ rvec;

    % Compute a coefficients
    a = zeros(n+1,1);
    for j = 1:n+1
        a(j) = y(j);
    end

    % Compute b and d coefficients
    b = zeros(n,1);
    d = zeros(n,1);

    for j = 1:n
        d(j) = (c(j+1) - c(j)) / (3*h(j));
        b(j) = (a(j+1) - a(j))/h(j) - (h(j)/3)*(2*c(j) + c(j+1));
    end
end
```

Problem 4 works with the following code:

```
coeffs=zeros(length(x)-1, 4);
for k=1:length(x)-1
    coeffs(k,:)=[d(k) c(k) b(k) a(k)];
end

breaks=[x];
pp=mkpp(breaks,coeffs);
ystar=ppval(pp,xstar);
```

At first, we're running `pp=mkpp(breaks,coeffs)` which takes the intervals over the x vector, and the coefficients, and builds cubic polynomials using the values d_i, c_i, b_i, and a_i for the interval between x_i and x_{i+1}. Running `mkpp` results in the output `pp` is a matlab struct with the following fields: `form`, `breaks`, `coefs`, `pieces`, `order`, and `dim`. `form` seems to be a constant string term with the value *pp* for piecewise polynomial. `breaks` represents the increasing list of x values of our points. `coefs` is a matrix of coefficients set up for the equation

$$f(x) = a(x - x_i)^3 + b(x - x_i)^2 + c(x - x_i) + d$$

Molality	0.005	0.010	0.020	0.050	0.100	0.200	0.500	1.000	2.000
Coefficient	0.924	0.896	0.859	0.794	0.732	0.656	0.536	0.430	0.316

TABLE 1. The Molality Given the Meah Activity Coefficients of Silver Nitrate

Days After Birth	0	6	10	13	17	20	28
avg. weight, *mg* (from young leaves)	7	18	45	40	32	30	30.5
avg. weight, *mg* (from mature leaves)	7	16	19	15	12	10.5	10

TABLE 2. Average weights of larvae from Feeny, P. (1970) *Ecology*, **51** (4), 565-581.

based on the i^{th} interval. `pieces` is the number of elements in the partition; `order` is the order of the polynomials; and `dim` is the dimensionality of the target. All of this then in turn gets sent to the function `ppeval(pp,xstar)`, which is a canned matlab function specifically for taking this data structure pp and generating y values given an arbitrary collection of x values. We were told to not edit this code, but curiously `xstar` is not defined, so we must in some fashion by defining it. It seems the best thing to do here is to take the minimum and maximum values from x and evenly subdivide the system for evaluation and to get a plot.

Problem 4 above adapted the original code from above such that the signature of the `naturalCubicSplines` function follows as `ystar=naturalCubicSplines(x,y,xstar)`. This is quite important for Problem 5 because we run the data in Table 1 through the method. We then set x to be Molality, and y to be Coefficient and $xstar = 0.032$ to see what our method outputs. We get $ystar = 0.8283$. The following code:

```
x = [0.005 0.010 0.020 0.050 0.100 0.200 0.500 1.000 2.000];
y = [0.924 0.896 0.859 0.794 0.732 0.656 0.536 0.430 0.316];

ystar = naturalCubicSpline1(x,y,0.032)
```

yields the following output:

```
>> gl3prob5

ystar =

    0.8283
```

Lastly note we have all this in thge file gl3prob5.m.

For problem 6 we consider Table 2 over which we will compute cubic spline interpolants. For portion (a) of Problem 6, we have the following code and output

```
daysAfterBirth  = [0 6 10 13 17 20 28];
weightYoungLeaves = [7 18 45 40 32 30.5 30];
weightMatureLeaves = [7 16 19 15 12 10.5 10];

youngWeight = naturalCubicSpline1(daysAfterBirth,weightYoungLeaves,3)
matureWeight = naturalCubicSpline1(daysAfterBirth,weightMatureLeaves,3)
meanWeight = (youngWeight + matureWeight)/2

youngWeightMatlab = spline(daysAfterBirth,weightYoungLeaves,3)
```

```
matureWeightMatlab = spline(daysAfterBirth,weightMatureLeaves,3)
meanWeightMatlab = (youngWeightMatlab + matureWeightMatlab)/2
```

So our mean weight works out to be $9.3754mg$ for our not-a-knot cubic spline while the matlab cubic spline outputs $3.8707mg$.

For portion (b) of problem 6 the fastest method for getting to where the maximum of our natural cubic splines lies in simply plotting and looking at the plots for the maximum, so we went about that. Our plot follows in Figure **??**

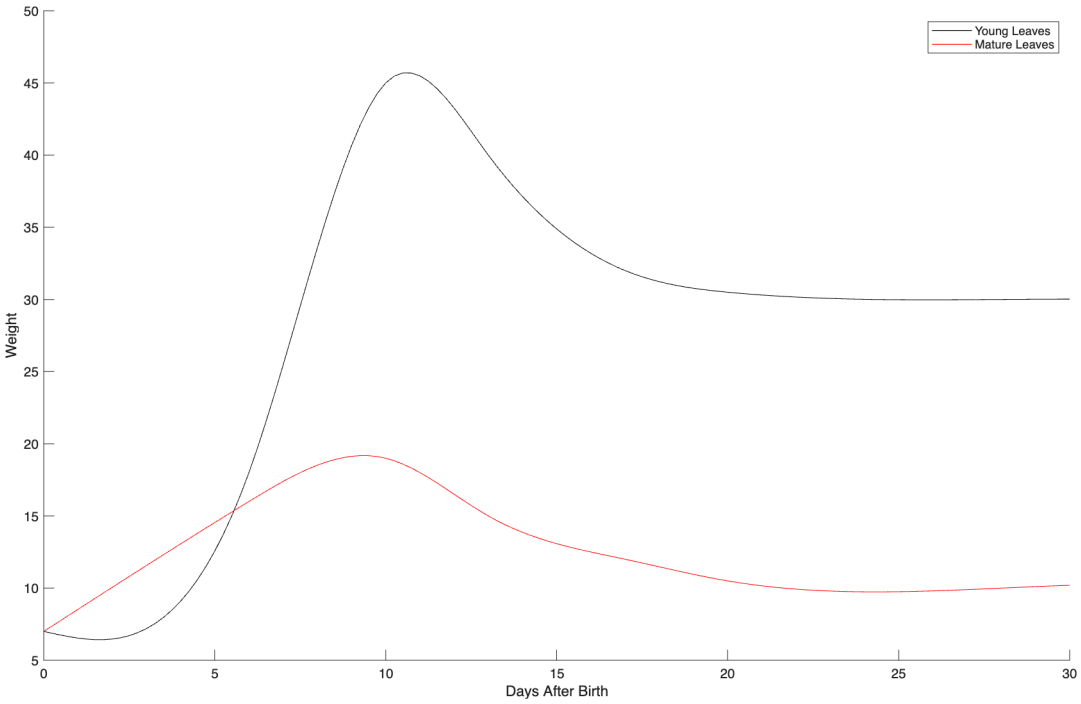

FIGURE 1. Weight of Young and Mature Leaves, in mg